Ir77PVAsset — Client API

Functions a client calls to fill PeregrineV, in call order

Startup (once)

Function	What it does
`SetPeregrineV(context)`	Links Asset to the PeregrineV context that owns the devices, allocators, layouts and pipelines. Every other call reads the current device from it. Call it before anything else.
`SetShaderRoot(path)`	Sets the folder the compiled `.spv` shader files are loaded from. It is optional; the default is the engine's `Shader` folder. Call it before `CreateShaders` if your shaders live elsewhere.
`CreateShaders()`	Loads every shader file found under the shader root for the current device. Missing required shaders (vertex, fragment, CEF) fail the call; optional ones (skinned, shadow, mesh) are skipped. Mesh shaders are only loaded when the device supports mesh shading. It records which pipeline kinds have all their stages loaded.
`GetPipelineKinds(kinds)`	Returns the pipeline kinds `CreateShaders` found complete for the current device. Pass this list into `CreatePipelines` so only buildable pipelines are created. It fails if `CreateShaders` has not been called.

Level load: textures and materials

Function	What it does
`CreateTexture(rgba, w, h, srgb, id, wait)`	Uploads one RGBA8 image (`w × h × 4` bytes) to the GPU and returns its texture id. Set `srgb` to true for color data such as albedo and emissive, and false for data maps such as normal, ORM and AO. The image has one mip

Function	What it does
	level and uses a shared linear, repeating sampler. With `wait = false` the call returns immediately and `Tick` frees the staging memory later.
`CreateMaterial(constants, [textures], id)`	Creates one material: a constants buffer plus a descriptor set with five texture slots. Slots are albedo, normal, ORM, emissive and AO; any slot left as `IR77PV_NO_TEXTURE` uses a placeholder (a flat normal for the normal slot). Texture ids must come from `CreateTexture`. Returns the material id used by every draw-item call.
`UploadMaterials(constants[], [textures[]], ids[])`	Creates a whole list of materials in one call. All the constants go to the GPU in a single upload instead of one per material. If a textures list is given it must be the same length as the constants list. Returns the material ids in the same order.

Level load: geometry

Each object in the scene is one indirect command: `firstIndex`/`indexCount` select its indices, `vertexOffset` is where its vertices start, and `firstInstance`/`instanceCount` select its slots in `models[]`. Sort objects by material so each material is one contiguous run of commands.

Function	What it does
`UploadScene(scene, scene_id, wait)`	Vertex path: uploads the stacked vertices, indices (static and skinned) and the indirect commands as one buffer per channel. The indirect buffer is per-frame host memory, so objects can be hidden, shown or resized later without re-uploading. Returns a scene id; uploading another scene does not replace the first. Use it on devices without mesh shading or for skinned geometry.
`UploadMeshlets(scene, meshlet_id, records, wait)`	Mesh path: takes the same stacked scene and

Function	What it does
	cuts each object's range into meshlets separately, so no meshlet ever mixes two objects. It writes one draw record and one mesh-task command per instance, and remembers where each object's records start. `records` returns the total record count. Fails on devices without mesh shading (fall back to `UploadScene`), and handles static vertices only.
`UploadMesh(input, mesh_id, wait)`	Uploads a single standalone mesh into its own vertex and index buffers. It is drawn with a direct draw through `MakeDrawItem`, not indirectly. Meant for debug, procedural or one-off geometry. Real levels should use `UploadScene` or `UploadMeshlets`.

Every frame

Function	What it does
`Tick(frame)`	Call once per frame, before Paint records, with the frame-in-flight index about to be recorded. It frees the staging memory of uploads that have finished on the GPU. It writes any show/hide or instance-count changes into this frame's copy of the indirect buffers. It also releases unloaded buffers once no in-flight frame can still read them.
`MakeCamera(eye, target, fov, aspect)`	Builds the camera block for set 0 binding 0: view, projection, view-projection and eye position. The projection is flipped for Vulkan's Y-down clip space and uses a 0–1 depth range. Pass the result to `paint->SetFrameData(w, Camera, ...)`. It is a static helper and changes nothing in Asset.
`MakeIndirectDrawItem(w, kind, scene_id, material, first_cmd, count, skinned, item)`	Builds one draw item for commands `[first_cmd, first_cmd + count)` of a vertex-path scene, normally one material run. It fills in the pipeline, layout, material set, vertex/index buffers and indirect range. Set

Function	What it does
	`skinned` to draw from the skinned channels. One item draws every object in the run with a single indirect call.
`MakeMeshletSceneDrawItem(w, meshlet_id, material, first_cmd, count, item)`	The mesh-path equivalent of `MakeIndirectDrawItem`, taking the same command indices. It maps those commands to their draw records and builds one mesh-task indirect draw for them. This is how one material run becomes one mesh draw.
`MakeDrawItem(w, kind, mesh_id, material, item)`	Builds a direct draw item for a mesh uploaded with `UploadMesh`. The caller sets `first_instance` to pick the model slot. It rejects the Mesh pipeline kind, which needs meshlets.
`MakeMeshletDrawItem` / `MakeMeshletIndirectDrawItem`	Lower-level mesh-path items. `MakeMeshletDrawItem` draws all meshlets of a single-object meshlet mesh directly. `MakeMeshletIndirectDrawItem` draws a raw range of draw records indirectly. Most clients use `MakeMeshletSceneDrawItem` instead.

Runtime changes

No re-upload; changes are applied to each frame's copy of the indirect buffer by `Tick`.

Function	What it does
`SetSceneObjectVisible(scene_id, cmd, on)`	Hides or shows one object and all its copies on the vertex path. Hiding sets the command's instance count to 0; showing restores it. The geometry and `models[]` slots are untouched.
`SetSceneObjectInstances(scene_id, cmd, count)`	Changes how many copies of an object are drawn on the vertex path, for spawning and despawning. The copies use `models[firstInstance .. firstInstance + count)`, so the client must reserve enough slots before the next object's range. If the object is hidden, the new count is applied when it is shown again.

Function	What it does
`SetMeshletObjectVisible(meshlet_id, cmd, on)`	Hides or shows one object and all its copies on the mesh path. Hiding sets each of its records' task-group count to 0; showing restores it. The meshlets and records stay in place.
`SetMeshletRecordVisible(meshlet_id, record, on)`	Hides or shows a single copy (one draw record) on the mesh path. The number of records is fixed when the meshlets are built. To have spawn capacity, upload spare instances and start them hidden.
`GetMeshletFirstRecord(meshlet_id, cmd, first)`	Returns the first draw record of an object on the mesh path. Copy `i` of that object is record `first + i`. Use it to find the record to pass to `SetMeshletRecordVisible`.

Unload

Remove the draw items from Paint first, and keep calling `Tick` so the buffers are actually released.

Function	What it does
`UnloadScene(scene_id)`	Retires a vertex-path scene's buffers. They are held until every frame that might still be reading them has finished, then freed. The scene id stays reserved and lookups for it fail afterwards.
`UnloadMeshlets(meshlet_id)`	Retires a mesh-path scene's buffers and its set 3 descriptor set. They are freed after the frames in flight, like `UnloadScene`. The id stays reserved and lookups for it fail afterwards.
`UnloadMesh(mesh_id)`	Retires a standalone mesh's vertex and index buffers. They are freed after the frames in flight. `GetMesh` and `MakeDrawItem` fail for that id afterwards.

Lookups (optional)

Function	What it does
`GetMesh(mesh_id, mesh)`	Returns the buffers and index count of an `UploadMesh` mesh. Fails for unknown or unloaded ids.
`GetMeshlets(meshlet_id, mesh)`	Returns the set 3 buffers, descriptor set and meshlet count of a meshlet scene. Fails for unknown or unloaded ids.
`GetMaterial(material_id, material)`	Returns a material's constants buffer and descriptor set. Fails for unknown ids.
`GetTextureInfo(texture_id, info)`	Returns the sampler, image view and layout of a texture, ready to bind to a descriptor. Fails for unknown ids.

Paint calls made alongside Asset, every frame

Function	What it does
`paint->SetFrameData(w, Camera, ...)`	Writes the camera block for window `w` into this frame's camera buffer (set 0 binding 0). Use the output of `MakeCamera`. Every pipeline reads it.
`paint->SetFrameData(w, Instances, models, ...)`	Writes one model matrix per instance slot into this frame's Instances buffer (set 0 binding 2). The vertex path finds its matrix through `gl_InstanceIndex`, the mesh path through each draw record's instance. The array must cover every slot the scene's commands use, up to the buffer's capacity.
`paint->SetDraws(w, pass, items)`	Replaces the draw list for one pass of window `w`. Pass the items built by the `Make*DrawItem` calls, normally one per material run. Removing an item here is the first step of unloading.

The testbed calls (`CreateTestbed`, `UpdateTestbed`) are development-only.